# DESARROLLO FULLSTACK III

***Grupo Cordillera – Plataforma de monitoreo inteligente para el desempeño organizacional***

***Fecha: 30/03/2026***

***Integrantes:***

***Alejandra Reyes***

***Sección 001D***

## Índice

1. Contextualización del caso
2. Selección de patrones de arquitectura
3. Comunicación entre microservicios: API Gateway
4. Arquitectura de microservicios, contenedores y consideraciones éticas
5. Evaluación general del diseño
6. Conclusión
7. Referencias bibliográficas

## 1. Contextualización del caso

Grupo Cordillera es una empresa del sector retail y comercialización de productos para el hogar y tecnología, con presencia en múltiples ciudades del país a través de una red de sucursales en expansión. En los últimos años, la organización ha experimentado un crecimiento sostenido, ampliando tanto su catálogo de productos como su cobertura geográfica. Sin embargo, dicho crecimiento ha traído consigo desafíos significativos en la gestión y análisis de la información.

Actualmente, la empresa opera con un conjunto heterogéneo de sistemas independientes: un sistema de punto de venta, una plataforma de comercio electrónico, un sistema de gestión de inventarios, herramientas de gestión financiera y un sistema de atención al cliente. Esta dispersión tecnológica implica que la información relevante para la toma de decisiones se encuentra fragmentada en múltiples plataformas, sin integración entre sí.

Como consecuencia, los equipos deben recurrir a procesos manuales para consolidar datos: extracción individual desde cada sistema, traspaso a hojas de cálculo, validación manual de indicadores y preparación de reportes para la alta dirección. Este flujo de trabajo genera retrasos en la disponibilidad de la información, riesgo de errores en la consolidación y una dependencia crítica de personas para producir reportes que, en muchos contextos, deberían ser automatizados.

Frente a esta problemática, la organización ha decidido desarrollar una plataforma tecnológica basada en microservicios que permita consolidar la información proveniente de los distintos sistemas internos y proporcionar herramientas de monitoreo en tiempo real. La solución contempla tres módulos principales: gestión de datos organizacionales, gestión de indicadores clave de desempeño (KPI) y visualización de reportes para la alta dirección.

## 2. Selección de patrones de arquitectura

Para abordar los requerimientos de Grupo Cordillera, se optó por una arquitectura de microservicios en lugar de una arquitectura monolítica. La siguiente tabla resume la justificación de esta decisión:

| Criterio | Monolítica | Microservicios ✓ |
|---|---|---|
| Estructura | Toda la app en un solo proyecto | Servicios independientes por dominio |
| Escalabilidad | Escala todo o nada | Escala cada servicio por separado |
| Acoplamiento | Alta dependencia entre módulos | Bajo acoplamiento entre servicios |
| Despliegue | Lento y riesgoso | Independiente y ágil |
| Mantenimiento | Difícil a largo plazo | Equipos paralelos por dominio |

Dado que Grupo Cordillera requiere consolidar información de múltiples sistemas independientes con distintos ritmos de actualización, la arquitectura de microservicios permite escalar y mantener cada módulo (datos, KPI, reportes) de forma autónoma sin afectar el funcionamiento global del sistema.

Dentro de esta arquitectura se implementan tres patrones de diseño principales:

### 2.1 Repository Pattern

El patrón Repository establece una capa de abstracción entre la lógica de negocio y el acceso a datos. En lugar de que cada microservicio consulte directamente las fuentes de datos, todas las operaciones se canalizan a través de repositorios especializados. Para el Microservicio de Datos Organizacionales, esto permite acceder de forma estandarizada a la información de ventas, inventarios y finanzas proveniente de los distintos sistemas internos de Grupo Cordillera, facilitando el

mantenimiento y la sustitución de tecnologías de persistencia sin impactar la lógica de negocio (Fowler, 2002).

### 2.2 Factory Method

El patrón Factory Method delega la creación de objetos a un método especializado, evitando instanciarlos directamente en el código cliente. En el Microservicio de KPI, distintos tipos de indicadores —ventas, logística, rentabilidad— requieren instancias con comportamientos diferentes. La fábrica determina qué tipo de KPI construir según los parámetros recibidos, manteniendo el principio de responsabilidad única y permitiendo incorporar nuevos tipos de indicadores sin modificar el código existente (Gamma et al., 1994).

### 2.3 Circuit Breaker

El patrón Circuit Breaker protege al sistema ante fallos en la comunicación entre microservicios. Opera en tres estados: cerrado (funcionamiento normal), abierto (fallo detectado, peticiones rechazadas o redirigidas) y semiabierto (prueba de recuperación). En la arquitectura de Grupo Cordillera, este patrón es crítico en la comunicación entre el API Gateway y los microservicios de datos: si el sistema de punto de venta presenta una falla, el panel de reportes continúa operando sin interrupciones, garantizando la disponibilidad del sistema ante fallos parciales (Newman, 2015).

---

## 3. Comunicación entre microservicios: API Gateway

Para gestionar la comunicación en la plataforma de Grupo Cordillera se implementa un API Gateway como único punto de entrada para todas las peticiones del cliente (panel gerencial web) hacia los microservicios del backend. En lugar de que el frontend deba conocer y gestionar conexiones individuales hacia cada servicio, interactúa exclusivamente con el Gateway, que se encarga internamente del enrutamiento. La clave de este diseño es que el cliente nunca sabe que existen múltiples servicios operando de forma independiente (Poblete, 2026).

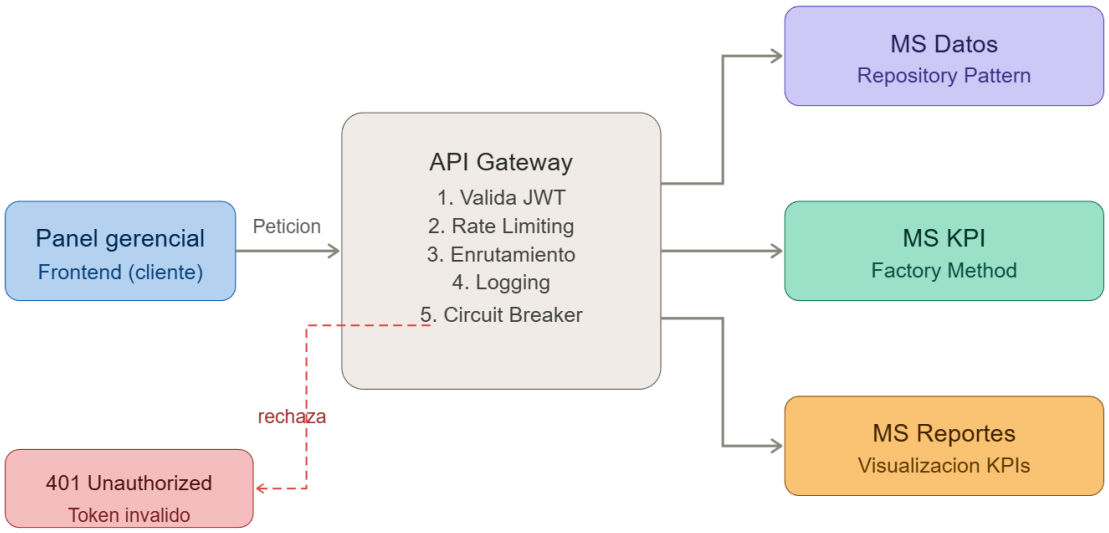


Figura 3: Flujo de comunicacion a traves del API Gateway

**Aporte a la eficiencia técnica**

El API Gateway centraliza el enrutamiento basándose en la URL y el método HTTP de cada petición, y aplica Rate Limiting mediante Bucket4j para controlar el número de solicitudes por cliente en un período determinado. Esto protege los servidores ante sobrecargas y garantiza disponibilidad equitativa para todos los usuarios que consulten reportes de forma simultánea. Adicionalmente, el Gateway centraliza el logging y monitoreo a través de herramientas como Grafana + Prometheus o Datadog, registrando cada solicitud y generando alertas automáticas cuando una métrica supera umbrales definidos, sin que los microservicios deban gestionar este aspecto.

**Aporte a la eficiencia operativa**

A nivel de seguridad, el API Gateway actúa como primer filtro del ecosistema mediante validación centralizada de tokens JWT (JSON Web Token). El flujo es el siguiente: (1) el cliente envía su solicitud con el token en el header Authorization: Bearer <token>; (2) el Gateway intercepta y valida la firma, expiración y permisos; (3) si el token es válido, la solicitud pasa al microservicio correspondiente; (4) si es inválido, el Gateway retorna un error 401 Unauthorized y el microservicio nunca se

ejecuta. Esto reduce la superficie de ataque y estandariza las políticas de acceso en un único punto.

---

## 4. Arquitectura de microservicios, contenedores y consideraciones éticas

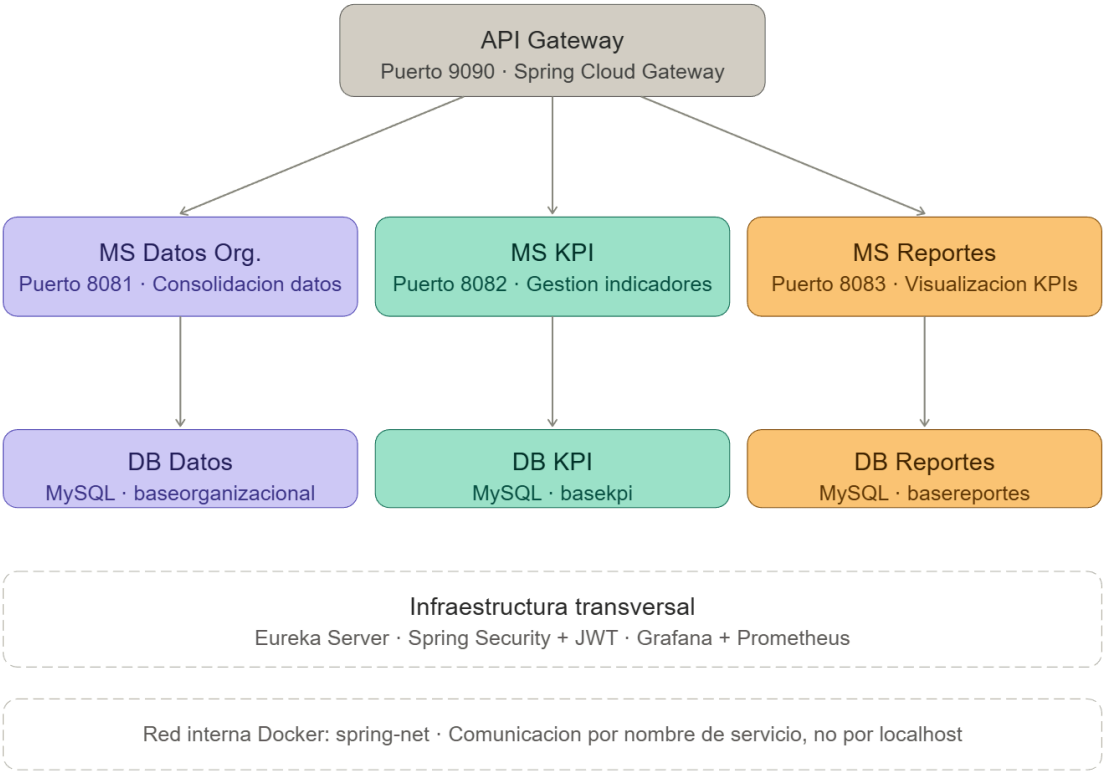


Figura 4: Arquitectura de microservicios — Grupo Cordillera

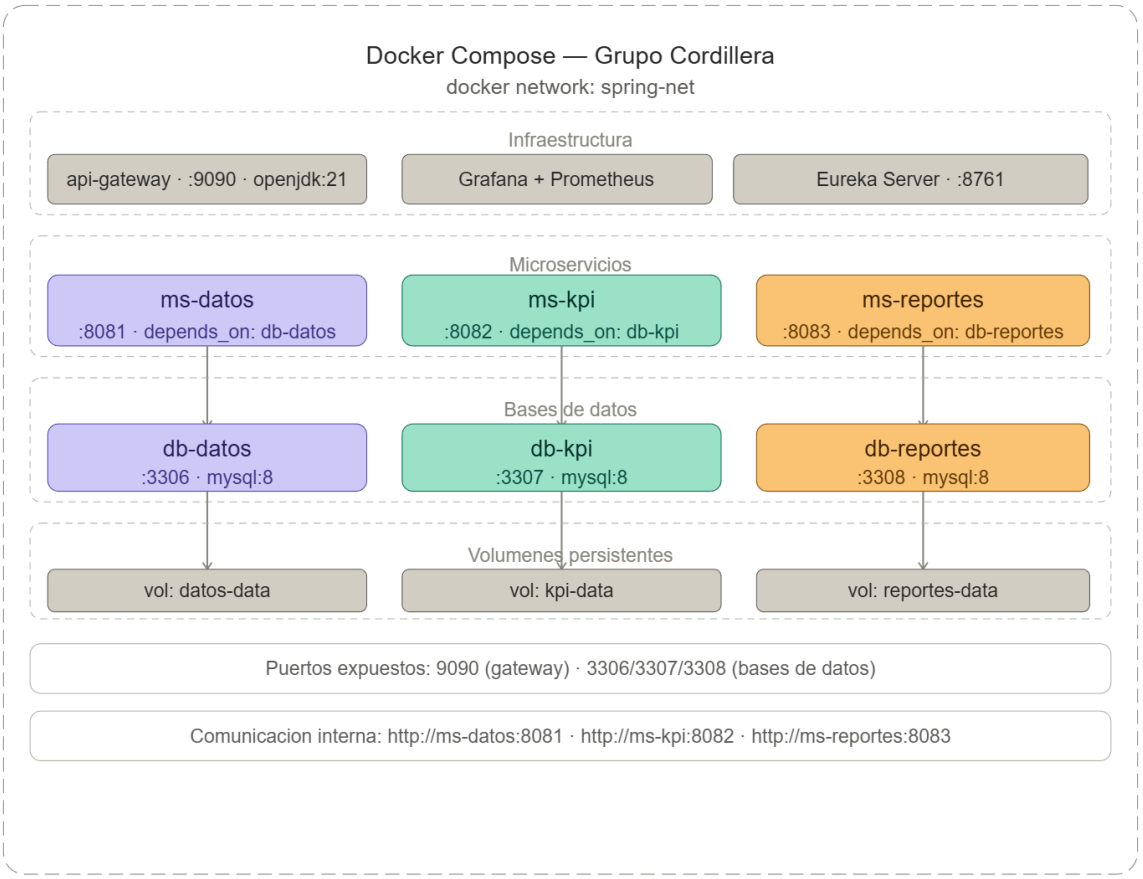


Figura 5: Diagrama de contenedores Docker — Grupo Cordillera

Cada microservicio se implementa con Spring Boot y JPA a través del Repository Pattern, orquestados con Docker Compose en una red interna denominada spring-net. El API Gateway opera en el puerto 9090 y los microservicios en los puertos 8081, 8082 y 8083, cada uno con su propia base de datos MySQL y volúmenes Docker para persistencia.

**Seguridad:** la autenticación se centraliza en el API Gateway mediante tokens JWT y Rate Limiting con Bucket4j, protegiendo la plataforma ante accesos no autorizados y ataques de denegación de servicio (Poblete, 2026).

**Privacidad:** el aislamiento de contenedores Docker garantiza que cada microservicio acceda únicamente a su propia base de datos, impidiendo que el MS Reportes, por ejemplo, acceda directamente a la base de datos de KPI. La información sensible de la organización no es accesible entre servicios sin pasar por el Gateway, que actúa como único punto de control.

**Sostenibilidad:** los contenedores Docker utilizan imágenes ligeras con multi-stage build, reduciendo el consumo de recursos computacionales. El despliegue independiente permite escalar únicamente los servicios con mayor carga, evitando el desperdicio de recursos que implicaría escalar toda una aplicación monolítica. A largo plazo, la arquitectura modular facilita la incorporación de nuevos módulos sin reescribir el sistema completo.

---

## 5. Evaluación general del diseño

La arquitectura propuesta responde de manera directa a los requerimientos funcionales definidos para Grupo Cordillera. El Microservicio de Datos Organizacionales centraliza la información dispersa en los distintos sistemas internos, eliminando la necesidad de consolidación manual en hojas de cálculo. El Microservicio de KPI permite definir y calcular indicadores estratégicos en tiempo real, reemplazando la validación manual actual. El Microservicio de Reportes proporciona el panel de control que la alta dirección requiere para tomar decisiones oportunas.

Desde el punto de vista técnico, la combinación del patrón Repository con JPA garantiza la persistencia estandarizada de datos; el Factory Method permite extender los tipos de KPI sin modificar código existente, cumpliendo el principio Open/Closed; y el Circuit Breaker asegura que un fallo en un sistema externo no paralice el panel ejecutivo. El API Gateway centraliza la seguridad, el control de tráfico y el monitoreo, evitando que cada microservicio deba implementar estas preocupaciones de forma independiente.

La solución es escalable: si en el futuro Grupo Cordillera requiere incorporar un módulo de logística o atención al cliente, puede hacerse desplegando un nuevo microservicio sin afectar los existentes. En síntesis, el diseño cumple con los requerimientos funcionales del cliente y las especificaciones técnicas establecidas para esta etapa del proyecto.

---

## 6. Conclusión

El diseño de arquitectura propuesto para Grupo Cordillera representa una solución técnica sólida y alineada con los requerimientos de la organización. La adopción de una arquitectura de microservicios, en combinación con los patrones Repository, Factory Method y Circuit Breaker, permite construir una plataforma modular, escalable y resiliente que reemplaza los procesos manuales de consolidación de datos por un sistema automatizado de monitoreo en tiempo real.

La incorporación del API Gateway como punto de entrada único, con validación JWT, Rate Limiting y monitoreo centralizado, garantiza que la plataforma opere de forma segura y controlada. La contenerización mediante Docker asegura la portabilidad y el aislamiento de cada componente, facilitando tanto el despliegue en producción como el mantenimiento a largo plazo.

Esta primera etapa sienta las bases arquitectónicas sobre las cuales se desarrollarán los componentes frontend y backend en las evaluaciones siguientes, avanzando hacia una solución integral para la gestión del desempeño organizacional de Grupo Cordillera.

---

## 7. Referencias bibliográficas


Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Newman, S. (2015). *Building microservices: Designing fine-grained systems*. O'Reilly Media.

Poblete, V. (2026). *Guía de arquitecturas de software — Fullstack III* [Material de cátedra]. DuocUC, Escuela de Informática y Telecomunicaciones.

Poblete, V. (2026). *API Gateway* [Material de cátedra]. DuocUC, Escuela de Informática y Telecomunicaciones.

Poblete, V. (2026). *Docker — Fullstack III* [Material de cátedra]. DuocUC, Escuela de Informática y Telecomunicaciones.